Implementation Prompt: Claw — WebSocket-Native Agentic Coding Harness in TypeScript

Mission

Build **Claw** — a TypeScript agentic coding harness that connects directly to the OpenAI Responses API over a persistent WebSocket connection. No HTTP agent loop. No proxy. No bridge. One WebSocket per agent. Multiple connections for parallel subagents.

This is a clean-room TypeScript implementation that synthesizes:

- **Claude Code's best patterns** (4-layer memory, self-healing continuation, progressive skill disclosure, subagent context isolation, skeptical-memory retrieval)
- **Codex CLI's strengths** (shell-centric tools, OS-level sandboxing, Responses API native loop)
- **OpenDev paper innovations** (5-stage adaptive compaction, eager construction, dual-memory anti-drift)

…running over a WebSocket transport to the OpenAI Responses API at `wss://api.openai.com/v1/responses`.

Before Writing a Single Line of Code — Required Research

Do all of the following first. Do not skip any step. These form the ground truth for every architectural decision.

1. Read the actual Claude Code source code

The real Claude Code TypeScript source is available at: `https://github.com/Darios-mom-used-npm-birth-force/claude-code-source-code-is-my-secret-sauce`

Fetch and read these files in full — they are your primary implementation reference:

File	Why it matters
`QueryEngine.ts`	The core engine class — one instance per conversation, `submitMessage()` is an async generator, session state persists across turns
`cli/transports/WebSocketTransport.ts`	Production-grade WS transport: `CircularBuffer` replay, sleep detection, ping/pong health, keepalive frames, exponential backoff with jitter, permanent close codes

File	Why it matters
`cli/transports/HybridTransport.ts`	Hybrid WS-read / HTTP-write transport: `SerialBatchEventUploader` pattern, 100ms stream-event batching
`commands/compact/compact.ts`	Three-layer compaction: session-memory first → reactive-compact → microcompact + summarize
`commands/memory/memory.tsx`	Real memory file management patterns
`constants/tools.ts`	All actual tool names: `FILE_READ_TOOL_NAME`, `FILE_EDIT_TOOL_NAME`, `GREP_TOOL_NAME`, `GLOB_TOOL_NAME`, `SKILL_TOOL_NAME`, `TOOL_SEARCH_TOOL_NAME`, cron tools, agent tools
`constants/systemPromptSections.ts`	`systemPromptSection` (cached) vs `DANGEROUS_uncachedSystemPromptSection` (volatile, breaks prompt cache)
`constants/common.ts`	`getSessionStartDate = memoize(getLocalISODate)` — captured once at session start for prompt cache stability
`Task.ts`	Task abstraction and lifecycle
`Tool.ts`	Tool interface, `ToolUseContext`, `toolMatchesName`

Read every one of those files completely before writing anything.

2. Read the reference Python implementations

Fetch these from `alankatanoisi/learn-claude-code`:

- `agents/s01_agent_loop.py` — core while-loop pattern
- `agents/s04_subagent.py` — context isolation via fresh message arrays
- `agents/s05_skill_loading.py` — frontmatter-only loading, on-demand body
- `agents/s06_context_compact.py` — three-layer compression pipeline
- `agents/s08_background_tasks.py` — background task queue
- `agents/s09_agent_teams.py` — multi-agent coordination
- `agents/s_full.py` — all mechanisms combined

These are the conceptual blueprint you are translating to TypeScript with a WebSocket transport.

3. Fetch official Responses API WebSocket documentation

Search for and read:

- `OpenAI Responses API WebSocket mode` — the `wss://api.openai.com/v1/responses` endpoint docs

- `response.create` input event schema: `previous_response_id`, `input`, `tools`, `instructions`, `model`, `reasoning`, `truncation`, `store`
- Server-sent events: `response.created`, `response.output_item.added`, `response.content_part.delta`, `response.function_call_arguments.delta`, `response.function_call_arguments.done`, `response.output_item.done`, `response.done`, `response.failed`
- `input_item.create` — how tool results are submitted under WebSocket mode
- The `previous_response_id` reconnection pattern
- Compaction / `type: "compaction"` items in the Responses API

4. Read the two prior research reports

Both files are in the workspace:

- `/home/user/workspace/websocket-native-agentic-harness.pplx.md` (1,291 lines, Parts 1-19)
- `/home/user/workspace/engineering-optimization-maximal-developer-utility.pplx.md` (550 lines, Parts 1-10)

These are the definitive specification. Read both in full before proceeding.

Project Structure

```
claw-ts/
├── package.json
├── tsconfig.json
├── .env.example
├── .gitignore
│
├── src/
│   ├── index.ts                    # Entry point / bin
│   ├── cli.ts                      # Arg parsing, session bootstrap
│   │
│   ├── transport/
│   │   ├── types.ts                # WebSocket state machine types
│   │   ├── circular-buffer.ts       # CircularBuffer<T> — port from real source
│   │   ├── websocket-transport.ts    # Core WS transport (port from real source)
│   │   └── connection-pool.ts       # Multi-connection pool for subagents
│   │
│   ├── engine/
│   │   ├── query-engine.ts          # QueryEngine class (port from real source)
│   │   ├── event-stream.ts          # Streaming event parser → AgentEvent
│   │   ├── continuation.ts          # Self-healing: truncation, retry, backoff
│   │   └── loop-detector.ts         # Anti-thrashing sliding window
│   │
│   ├── context/
│   │   ├── system-prompt-section.ts  # Cached vs volatile sections (from real source)
│   │   ├── prompt-builder.ts        # CLAW.md hierarchy + skill index assembly
│   │   ├── compaction/
│   │   │   ├── micro-compact.ts     # Layer 1: micro-compact older tool results
│   │   │   ├── session-memory.ts    # Layer 2: session-memory compaction
```

```
│   │   │   └── full-compact.ts       # Layer 3: full LLM summarization (ACC 5-stage)
│   │   └── tool-result-shrinker.ts   # Per-tool result compression
│   │
│   ├── memory/
│   │   ├── layer-manager.ts          # Orchestrates all 4 memory layers
│   │   ├── static.ts                 # Layer 1: CLAW.md loader (skeptical-hint format)
│   │   ├── auto-memory.ts            # Layer 2: Agent-written per-session memory
│   │   ├── auto-dream.ts             # Layer 3: Background consolidation
│   │   └── semantic-search.ts        # Embedding + grep hybrid retrieval
│   │
│   ├── skills/
│   │   ├── loader.ts                 # Frontmatter-only at startup, full body on demand
│   │   └── registry.ts               # Skill index map
│   │
│   ├── tools/
│   │   ├── registry.ts               # Tool registry + dispatch
│   │   ├── shell.ts                  # Sandboxed shell executor (primary tool)
│   │   ├── file-read.ts              # Read with line limits
│   │   ├── file-edit.ts              # Exact-string replacement (matches FileEditTool)
│   │   ├── file-write.ts             # Atomic new-file write
│   │   ├── grep.ts                   # Regex search with context
│   │   ├── glob.ts                   # File pattern matching
│   │   ├── apply-patch.ts            # Unified diff with colorized review
│   │   ├── todo-write.ts             # TodoWrite tool (matches real source)
│   │   ├── task.ts                   # Subagent spawner tool
│   │   ├── skill-tool.ts             # Skill loader tool
│   │   ├── memory-write.ts           # Memory write tool
│   │   ├── tool-search.ts            # MCP tool search (lazy discovery)
│   │   └── compact.ts                # Manual compaction trigger
│   │
│   ├── safety/
│   │   ├── permission-engine.ts      # 3-level approval (manual/semi/auto)
│   │   ├── danger-rules.ts           # Non-overridable block patterns
│   │   ├── path-validator.ts         # Workspace escape prevention
│   │   └── hooks.ts                  # PreToolUse / PostToolUse lifecycle
│   │
│   ├── session/
│   │   ├── manager.ts                # previous_response_id threading, usage tracking
│   │   ├── persistence.ts            # JSONL transcript (atomic writes)
│   │   ├── checkpoint.ts             # Git commits + progress files
│   │   └── reconnect.ts              # Preemptive reconnect at 55 minutes
│   │
│   ├── ui/
│   │   ├── repl.ts                   # Main REPL (<300 lines, no monolith)
│   │   ├── renderer.ts               # Streaming ANSI renderer
│   │   ├── diff-display.ts           # Colorized diff review for apply_patch
│   │   └── status-bar.ts             # Single-line status (tokens, model, phase)
│   │
│   └── types/
│       ├── responses-api.ts          # OpenAI Responses API event types
│       ├── agent.ts                  # Internal agent state
│       ├── tools.ts                  # Tool call/result types
│       └── memory.ts                 # Memory layer types
│
├── skills/
│   └── agent-builder/SKILL.md
```

```
│
└── tests/
    ├── transport.test.ts
    ├── compaction.test.ts
    ├── memory.test.ts
    └── tools.test.ts
```

Package Setup

```
{
  "name": "claw-ts",
  "version": "0.1.0",
  "description": "WebSocket-native agentic coding harness",
  "type": "module",
  "bin": { "claw": "./dist/index.js" },
  "scripts": {
    "dev": "tsx src/index.ts",
    "build": "tsc",
    "test": "vitest run"
  },
  "dependencies": {
    "ws": "^8.18.0",
    "gray-matter": "^4.0.3",
    "diff": "^8.0.3",
    "dotenv": "^16.4.5",
    "chalk": "^5.3.0",
    "yaml": "^2.4.1",
    "zod": "^3.23.8",
    "simple-git": "^3.25.0",
    "lodash-es": "^4.17.21"
  },
  "devDependencies": {
    "@types/ws": "^8.5.12",
    "@types/node": "^20.0.0",
    "@types/diff": "^7.0.2",
    "@types/lodash-es": "^4.17.12",
    "tsx": "^4.21.0",
    "typescript": "^5.5.0",
    "vitest": "^1.0.0"
  }
}
```

`tsconfig.json`: `"strict": true`, `"module": "ESNext"`, `"moduleResolution": "bundler"`, `"target": "ES2022"`.

Module Specifications — Implement in This Order

Module 1: CircularBuffer (`src/transport/circular-buffer.ts`)

Port this directly from the real Claude Code source (`WebSocketTransport.ts` imports `CircularBuffer`). A fixed-capacity circular buffer that overwrites the oldest entry on overflow.

```
export class CircularBuffer<T> {
  private buffer: T[]
  private head = 0
  private size = 0
  private capacity: number

  constructor(capacity: number)
  add(item: T): void        // Overwrites oldest on overflow
  addAll(items: T[]): void
  toArray(): T[]            // Returns items in insertion order
  clear(): void
  get length(): number
}
```

This is used by the WebSocket transport for outbound message buffering and replay on reconnection. Every message with a UUID is buffered here so it can be replayed if the connection drops before the server acknowledges it.

Module 2: WebSocket Transport (`src/transport/websocket-transport.ts`)

This is the most critical module. Port and adapt from the real Claude Code source at `cli/transports/WebSocketTransport.ts`. Study that file completely — it is production-hardened and handles every edge case you will encounter.

Key constants to replicate exactly (they are the result of production experience, not guesses):

```
const KEEP_ALIVE_FRAME = '{"type":"keep_alive"}\n'
const DEFAULT_MAX_BUFFER_SIZE = 1000           // Message replay buffer capacity
const DEFAULT_BASE_RECONNECT_DELAY = 1000      // 1s base for exponential backoff
const DEFAULT_MAX_RECONNECT_DELAY = 30000      // 30s max delay between reconnects
const DEFAULT_RECONNECT_GIVE_UP_MS = 600_000   // 10 minutes before giving up
const DEFAULT_PING_INTERVAL = 10000            // Ping every 10s for health check
const DEFAULT_KEEPALIVE_INTERVAL = 300_000     // Keep-alive data frame every 5 minutes
const SLEEP_DETECTION_THRESHOLD_MS = DEFAULT_MAX_RECONNECT_DELAY * 2 // 60s
```

Permanent close codes — stop retrying immediately on these (server has definitively ended the session):

```
const PERMANENT_CLOSE_CODES = new Set([
  1002, // protocol error / handshake rejected (e.g. session reaped)
  4001, // session expired / not found
```

```
  4003, // unauthorized (can be retried ONLY if refreshHeaders returns new token)
])
```

State machine:

```
type WebSocketTransportState = 'idle' | 'connected' | 'reconnecting' | 'closing' | 'close
```

Class: `WebSocketTransport`

Constructor args:

- `url: URL` — the WebSocket endpoint
- `headers: Record<string, string>` — auth headers
- `sessionId?: string` — optional session ID for logging
- `refreshHeaders?: () => Record<string, string>` — called before each reconnect to get fresh auth tokens
- `options?: { autoReconnect?: boolean }` — defaults to true

Key implementation details from the real source (replicate these exactly):

Connection (`connect()`):

- Use the `ws` npm package in Node.js, `globalThis.WebSocket` in Bun
- Store event handlers as class-property arrow functions (not inline closures) so they can be removed in `doDisconnect()` — prevents listener accumulation across reconnects
- Record `connectStartTime = Date.now()` before opening — used for latency logging
- On open: record `lastActivityTime`, start ping interval, start keepalive interval
- Send `X-Last-Request-Id` header on reconnect so server can tell you what it already received

Message buffering and replay (`replayBufferedMessages(lastId: string)`):

- On reconnect, server sends back `X-Last-Request-Id` (via upgrade response headers in `ws` package)
- Find that ID in the circular buffer, evict confirmed messages, replay only unconfirmed ones
- Do NOT clear the buffer after replay — messages stay buffered until the server confirms on the next reconnect
- In Bun (no upgrade headers): replay everything; server deduplicates by UUID

Sleep detection — two places where this matters:

1. In `handleConnectionError()`: if `now - lastReconnectAttemptTime > SLEEP_DETECTION_THRESHOLD_MS`, the machine slept — reset reconnect budget and try fresh
2. In `startPingInterval()`: if the gap between consecutive tick callbacks `> SLEEP_DETECTION_THRESHOLD_MS`, the process was suspended — force reconnect immediately without waiting for ping/pong (because `ws.ping()` on a dead socket returns immediately with no error)

Exponential backoff with ±25% jitter:

```
const baseDelay = Math.min(
  DEFAULT_BASE_RECONNECT_DELAY * Math.pow(2, reconnectAttempts - 1),
  DEFAULT_MAX_RECONNECT_DELAY,
)
const delay = Math.max(0, baseDelay + baseDelay * 0.25 * (2 * Math.random() - 1))
```

Ping/pong health check:

- Every `DEFAULT_PING_INTERVAL` (10s), send a WebSocket ping
- If no pong received since last ping, call `handleConnectionError()` — connection is dead
- `pongReceived` flag is set to `true` on pong, `false` before each ping

Keepalive data frames:

- Every `DEFAULT_KEEPALIVE_INTERVAL` (5 minutes), send `KEEP_ALIVE_FRAME` as a data frame
- This resets proxy idle timers (Cloudflare drops connections after 5 minutes of inactivity)
- Do NOT do this in CCR (remote) sessions — they use their own heartbeat mechanism

`write(message: OutboundMessage): Promise<void>`:

- If message has a UUID, add to circular buffer and update `lastSentId`
- If state is not `connected`: silently drop (message is buffered for replay when connection restores)
- Serialize as `JSON.stringify(message) + '\n'` and call `sendLine()`
- Update `lastActivityTime` on every send

Listener cleanup (`removeWsListeners(ws)` + `doDisconnect()`):

- ALWAYS remove all event listeners from the old WebSocket object before closing it
- Without this, each reconnect orphans the old WS + its closures until GC

Adapting for Responses API: The real source's transport writes Claude Code's internal protocol messages. Yours writes OpenAI Responses API events. The transport structure is identical — only the message types change. Replace `StdoutMessage` with your `ResponsesApiInputEvent` type. The WebSocket URL changes from Claude Code's bridge URL to `wss://api.openai.com/v1/responses`. The auth header changes from Claude Code's session token to `Authorization: Bearer ${OPENAI_API_KEY}`.

Module 3: Responses API Types (`src/types/responses-api.ts`)

Define complete TypeScript types for every Responses API input event and server-sent event. Research these from the official docs before writing them.

Input events (what you send to the server):

```
type ResponseCreateEvent = {
  type: 'response.create'
  response: {
    model: string
    instructions?: string        // System prompt — injected once, cached server-side
    input: InputItem[]           // User messages, tool results
    tools?: ToolDefinition[]
    previous_response_id?: string // Threads state across turns
    reasoning?: { effort: 'low' | 'medium' | 'high' }
    truncation?: 'auto' | 'disabled'
    store?: boolean
    max_output_tokens?: number
  }
}

type InputItemCreateEvent = {
  type: 'input_item.create'
  item: {
    type: 'function_call_output'
    call_id: string
    output: string              // Tool result — this is how tool outputs are submitted
  }
}
```

Server-sent events (what you receive from the server): Define types for all of: `response.created`, `response.output_item.added`, `response.content_part.added`, `response.content_part.delta`, `response.content_part.done`, `response.function_call_arguments.delta`, `response.function_call_arguments.done`, `response.output_item.done`, `response.done`, `response.failed`, `error`.

Module 4: Event Stream Parser (`src/engine/event-stream.ts`)

An async generator that consumes raw Responses API server-sent events and emits higher-level `AgentEvent` objects. Maintains state for:

- Accumulating text deltas into complete content parts
- Accumulating function call argument deltas into complete tool call arguments (keyed by `call_id`)
- Tracking the current response ID

```
type AgentEvent =
  | { type: 'text_delta'; text: string }
  | { type: 'tool_call_ready'; callId: string; name: string; arguments: unknown }
  | { type: 'turn_complete'; responseId: string; usage: TokenUsage }
  | { type: 'error'; error: ResponsesApiError }

async function* parseEventStream(
  rawEvents: AsyncGenerator<ResponsesApiServerEvent>
): AsyncGenerator<AgentEvent>
```

When `response.function_call_arguments.done` fires, parse the accumulated argument string as JSON, validate it against the tool's schema, and emit `tool_call_ready`. If JSON parsing fails, emit a recovery signal (handled by the continuation module).

Module 5: System Prompt Section Cache (`src/context/system-prompt-section.ts`)

Port the system prompt section cache from the real source's `constants/systemPromptSections.ts`. This is how Claude Code maintains prompt cache stability — static sections are computed once and reused across turns, volatile sections recompute every turn and may break the cache.

```
type ComputeFn = () => string | null | Promise<string | null>

type SystemPromptSection = {
  name: string
  compute: ComputeFn
  cacheBreak: boolean
}

// Cached — computed once per session. Safe for static content.
export function systemPromptSection(name: string, compute: ComputeFn): SystemPromptSec

// Volatile — recomputes every turn. BREAKS THE PROMPT CACHE when value changes.
// Only use when real-time accuracy is more valuable than cache efficiency.
export function DANGEROUS_uncachedSystemPromptSection(
  name: string,
  compute: ComputeFn,
  reason: string // Force callers to justify the cache cost
): SystemPromptSection

export async function resolveSystemPromptSections(
  sections: SystemPromptSection[]
): Promise<(string | null)[]>

export function clearSystemPromptSections(): void // Called on /clear and /compact
```

Use this to build the system prompt in `prompt-builder.ts`. Each CLAW.md layer, each skill index entry, and each tool description should be a `systemPromptSection`. The date should use `getSessionStartDate = memoize(() => new Date().toISOString().split('T')[0])` — captured once at session start, never recomputed mid-session.

Module 6: QueryEngine (`src/engine/query-engine.ts`)

The core engine class. Port from the real source's `QueryEngine.ts`, replacing:

- Anthropic API calls → OpenAI Responses API WebSocket calls
- `Messages.create` → `client.sendCreate()` on the WebSocket transport
- Claude-specific types → Responses API types

Key architecture from the real source:

- One `QueryEngine` per conversation (not per turn)
- `mutableMessages: Message[]` — the conversation history, mutated in-place across turns
- `totalUsage: NonNullableUsage` — accumulated across all turns in the session
- `discoveredSkillNames = new Set<string>()` — cleared at start of each `submitMessage`, tracks skills used this turn
- `loadedNestedMemoryPaths = new Set<string>()` — persists across turns (never re-load same memory file)

`async *submitMessage(prompt: string): AsyncGenerator<SDKMessage>:`

This is an async generator. It yields events to the UI/caller as they happen:

1. Build system prompt via `resolveSystemPromptSections()`
2. Process input (slash commands, attachments) via `processUserInput()`
3. Persist user message to transcript BEFORE entering the query loop (so sessions are resumable even if killed mid-request)
4. `yield` a system init message with tool list, model, permission mode
5. Enter the query loop: `for await (const event of query(...))`
6. On each event: push to `mutableMessages`, persist to transcript, `yield*` normalized event to caller
7. On `result`: yield final result with total cost, usage, and stop reason

From the real source — `processUserInput` before the query loop:
The real code calls `processUserInput()` which handles slash commands. If the command is local (`shouldQuery = false`), return the local command output immediately without calling the API. If `shouldQuery = true`, proceed to the API loop. Mirror this pattern with your `/` command dispatch.

Transcript persistence (fire-and-forget for assistant messages, awaited for user/boundary messages):

```
// From the real source: fire-and-forget for assistant messages
// Awaiting blocks the generator, preventing message_delta from running
if (message.type === 'assistant') {
  void recordTranscript(messages)
} else {
  await recordTranscript(messages)
}
```

Module 7: Self-Healing Continuation (`src/engine/continuation.ts`)

When the model output is interrupted — token limit, API error, malformed tool call — the harness heals itself and continues. The model should NEVER need to see an error message unless a human decision is genuinely required.

Healing strategies by error type:

Error	Strategy
`context_length_exceeded`	Trigger ACC compaction immediately, then re-send the last user message
Response truncated (finish_reason = `length`)	Inject invisible continuation: `submitToolOutput(lastCallId, "continue")` — no apology, just continue
Malformed tool call JSON	Re-send with error context: `"Your last tool call had invalid JSON arguments: ${error}. Please try again with valid JSON."`
`rate_limit_exceeded`	Exponential backoff: 1s → 2s → 4s → 8s → 16s, then surface to user
`server_error`	Backoff: 2s → 5s → 10s, then surface to user
Parse error (arguments.done)	Re-prompt: `"The tool call arguments could not be parsed. Please reformulate the tool call."`

Track `retryCount` per turn. After 5 consecutive auto-retries, stop and show: `[claw] Auto-retry limit reached. Type /retry to try again or /compact to free up context.`

The continuation message (for truncation) must be injected as a `input_item.create` event on the existing WebSocket connection, not as a new `response.create`. This preserves the session state and costs only the new tokens, not the full re-send.

Module 8: Loop Detector (`src/engine/loop-detector.ts`)

Maintains a sliding window of the last 10 tool calls. Detects when the agent is thrashing on the same operation.

```
class LoopDetector {
  private window: Array<{ name: string; argHash: string }> = []
  private readonly windowSize = 10
  private readonly thrashThreshold = 3

  record(name: string, args: unknown): void
  isLooping(): boolean // true if same (name, hash) appears ≥ 3 times in window
  reset(): void
}
```

When `isLooping()` returns true, the tool handler feeds back an intervention message instead of executing the tool: `"You appear to be repeating the same operation. Stop, re-read the relevant file, re-read any error messages, and try a different approach."` — then marks the tool call as handled.

Module 9: Three-Layer Compaction

Port and adapt from the real source's `commands/compact/compact.ts`. The real source reveals a three-layer pipeline executed in this exact order:

Layer 1: Micro-compact (`src/context/compaction/micro-compact.ts`)

Run before all other compaction. Replace older tool results (beyond the last N) with compact placeholders. From the real source, `microcompactMessages` is called BEFORE the full `compactConversation`. Preserve results from expensive tools (like `file-read`).

```
const KEEP_RECENT = 5           // Preserve last 5 tool results verbatim
const PRESERVE_TOOLS = new Set(['file_read', 'glob', 'grep']) // Never shrink these

function microCompact(messages: Message[]): Message[] {
  // Walk tool results, replace anything older than KEEP_RECENT with:
  // "[Previous: used ${toolName}]"
}
```

Layer 2: Session-Memory Compaction (`src/context/compaction/session-memory.ts`)

Try this before full compaction. The real source calls `trySessionMemoryCompaction()` first — if it succeeds, skip the expensive full compaction. Session memory compaction works by:

1. Extracting key facts from the conversation using a lightweight LLM call
2. Writing those facts to `.claw/session-memory.md`
3. Replacing the conversation with a pointer to that file

This is much cheaper than full summarization and works for most cases.

Layer 3: Full Compaction with 5-Stage ACC (`src/context/compaction/full-compact.ts`)

The nuclear option. Implement the OpenDev paper's 5-stage Adaptive Context Compaction (ACC). Monitor `inputTokens` from the Responses API's `response.done` event. Apply stages progressively:

```
enum CompactionStage {
  Normal = 0,
  Warning = 1,          // > 70% context filled
  ObservationMask = 2,  // > 80%: older tool results → "[offloaded: ${toolName}]"
  FastPrune = 3,        // > 85%: remove oldest messages beyond recency window
  AggressiveMask = 4,   // > 90%: shrink recency window further
  FullCompaction = 5,   // > 99%: LLM summarize, replace history, write progress file
}
```

For full compaction:

1. Write progress file to `.claw/progress.md` (what was being done, files modified, next steps, blockers)
2. Write session transcript to `.claw/transcripts/session-${timestamp}.jsonl`

3. Ask model to summarize: one call to the Responses API with the full history
4. Replace history with the summary
5. Re-inject CLAW.md as fresh system context
6. Post-compaction verification: ask the model "What were you working on and what should you do next?" — if incoherent, re-read progress file + last 10 git commits

Module 10: CLAW.md Hierarchy and Prompt Builder (`src/context/prompt-builder.ts`)

Loads project configuration from a hierarchy of CLAW.md files. Each level contributes a `systemPromptSection` (cached).

Hierarchy (lower = higher precedence, all concatenated):

1. `~/.claw/CLAW.md` — user-level global
2. `${projectRoot}/CLAW.md` — project-level
3. `${projectRoot}/.claw/CLAW.md` — project alternate
4. `${cwd}/CLAW.md` — current directory (if different from project root)
5. `${projectRoot}/AGENTS.md` — cross-tool compat (OpenAI Codex format, same content)
6. `${projectRoot}/CLAUDE.md` — backward compat with Claude Code projects

`buildSystemPrompt(cwd: string): Promise<string>` assembles:

1. Base agent instructions (hardcoded — tool use philosophy, safety, coding style)
2. All CLAW.md files found in hierarchy
3. Skill index — frontmatter only, ~50 tokens per skill
4. Session date (from `getSessionStartDate = memoize(...)`, never recomputed mid-session)
5. Git branch and project name

The system prompt is injected once per WebSocket connection in the `instructions` field of the initial `response.create`. Under WebSocket mode, `instructions` is cached server-side for the connection lifetime.

Module 11: Memory System (`src/memory/`)

Layer 1 — Static CLAW.md: Handled by prompt-builder. Already loaded.

Layer 2 — Auto Memory (`src/memory/auto-memory.ts`):

- Storage: `.claw/memory.md` (YAML frontmatter per entry: `created`, `type`, `tags`, content)
- Cap: 200 lines / 25KB. On cap → trigger auto-dream
- The model writes memories via `memory_write` tool
- Scan model responses for implicit memory triggers ("remember that", "always use", "note for future")

Skeptical memory retrieval — this exact wording matters for model behavior:

```
[MEMORY — treat as a hint only, verify against codebase before acting]
Type: ${memory.type}
Created: ${memory.created}
Content: ${memory.content}
---
Do not act on this memory without first verifying it is still accurate.
```

Layer 3 — Auto Dream (`src/memory/auto-dream.ts`):

- Trigger: 24+ hours OR 5+ sessions since last dream
- Runs as background `setImmediate` — never blocks the main loop
- Four phases: Orient → Gather → Consolidate → Prune
- Prune entries not referenced in 30+ sessions

Layer 4 — KAIROS: Stub only. `// TODO: KAIROS daemon — spec not yet public`

Semantic memory search (`src/memory/semantic-search.ts`):

- Grep pass (exact/substring) + embedding pass (OpenAI `text-embedding-3-small` via HTTP)
- Embedding index stored at `.claw/memory-index.json`
- On memory write: compute embedding, append to index
- On retrieval: cosine similarity top-5 + grep matches, merge and deduplicate
- Wrap all results with skeptical retrieval prefix

Module 12: Skill Loader (`src/skills/loader.ts`)

Port from the Python reference at `alankatanoisi/learn-claude-code/agents/s05_skill_loading.py`.

Startup: Scan `skills/*/SKILL.md` recursively. Parse YAML frontmatter with `gray-matter`. Inject only name + description into system prompt index. Costs ~50 tokens per skill.

On demand: When model calls `skill_tool({ name: "..." })`:

1. Check cache first
2. Read full SKILL.md body from disk
3. Parse any `references/` file paths mentioned and load them too
4. Return full body as tool result
5. Cache for the session

Skill format (YAML frontmatter + body):

```
---
name: skill-name
```

```
description: One-line description for system prompt index (~10 words)
triggers:
  - keyword1
  - keyword2
---
# Full skill instructions here
```

Port all existing skills from `alankatanoisi/learn-claude-code/skills/` into `claw-ts/skills/`, updating any Python-specific references.

Module 13: Tool Registry and Tools (`src/tools/`)

Mirror the real Claude Code tool names from `constants/tools.ts`. Use the same conceptual names even though the implementations are yours:

Your tool	Mirrors CC tool	Purpose
`shell`	`SHELL_TOOL_NAMES`	Primary shell executor
`file_read`	`FILE_READ_TOOL_NAME`	Read with line limits, preserve in compaction
`file_edit`	`FILE_EDIT_TOOL_NAME`	Exact-string replacement (not patch)
`file_write`	`FILE_WRITE_TOOL_NAME`	Atomic new-file creation
`grep`	`GREP_TOOL_NAME`	Regex search with context lines
`glob`	`GLOB_TOOL_NAME`	File pattern matching
`apply_patch`	(custom)	Unified diff with colorized review
`todo_write`	`TODO_WRITE_TOOL_NAME`	Task tracking
`task`	`AGENT_TOOL_NAME`	Spawn isolated subagent
`skill_tool`	`SKILL_TOOL_NAME`	Load full skill body
`memory_write`	(custom)	Write to auto-memory layer
`tool_search`	`TOOL_SEARCH_TOOL_NAME`	Lazy MCP tool discovery
`compact`	(maps to `/compact`)	Manual compaction trigger

Shell tool — primary executor, sandboxed:

- Block dangerous patterns via `danger-rules.ts` BEFORE permission check
- Run via `child_process.spawn` (not `exec`)
- Timeout: 120s default, configurable per call
- Truncate output at 50,000 chars
- Workspace escape prevention: reject any command containing `../` sequences that resolve outside `cwd`

File edit tool — exact-string replacement (matches the real `FileEditTool` pattern):

```
// The real CC FileEditTool uses exact-string replacement, not patch
schema = {
  name: 'file_edit',
  parameters: {
    path: string,
    old_string: string,   // Must be unique in the file
    new_string: string
  }
}
```

apply_patch tool — unified diff with review:

- Parse with the `diff` package
- Display colorized review (red deletions, green additions, file path header)
- Check permission mode for auto/manual apply
- Shadow git: after every successful patch, `git add -A && git stash` in `.claw/shadow-git/`
- Return: `✓ Applied: ${description} (+${additions}/-${deletions})`

Tool result shrinker — applied to every tool result before it enters the conversation history:

Tool	Shrink behavior
`file_read`	Preserve full content — never shrink
`file_write`	`✓ Wrote ${bytes} bytes to ${path}`
`file_edit`	`✓ Edited ${path}: replaced ${n} chars`
`apply_patch`	`✓ Applied patch to ${path} (+${add}/-${del})`
`shell`	Truncate at 50K chars; if >1K also prepend line count
`grep`	Truncate at 10K chars
`memory_write`	`✓ Saved memory: ${preview}`
`task`	Pass through subagent summary as-is

Module 14: Subagent Spawner (`src/engine/` + `src/tools/task.ts`)

The core context isolation mechanism. From the Python reference at `agents/s04_subagent.py`: the subagent starts with empty messages, works independently, returns only its final summary. The parent's context is never polluted.

Implementation:

1. Create a new `WebSocketTransport` instance (separate WebSocket connection)
2. Build a fresh subagent system prompt: `"You are a coding subagent. Complete the given task. Return a concise summary of what you found or did."`
3. Run a complete `QueryEngine` instance with `mutableMessages = []`

4. Give the subagent only the tools specified in the task request (default: read-only tools)
5. When the subagent's loop completes (no more tool calls), extract its final text response
6. Close the subagent's WebSocket connection
7. Return the final text as the tool result for the `task` call

Connection pool (`src/transport/connection-pool.ts`): Manage up to 4 parallel subagent connections. When a subagent completes, its connection is closed (not pooled — each subagent gets a fresh connection for clean state).

Module 15: Permission Engine (`src/safety/permission-engine.ts`)

Three approval tiers. Check order (always):

1. **Danger rules** (non-overridable, checked first)
2. **Persistent allow rules** (previously approved patterns, stored in session)
3. **Permission mode** (manual / semi / auto)

Semi-auto (the recommended default):

- Auto-approve: `file_read`, `glob`, `grep`, `tool_search`, `shell` with read-only commands
- Prompt for approval: `file_write`, `file_edit`, `apply_patch`, `shell` with write operations

On prompt: `Apply? [y] yes [n] no [a] always allow this pattern [s] always skip`

Danger rules (`src/safety/danger-rules.ts`) — always blocked regardless of mode:

```
const ALWAYS_BLOCK = [
  /rm\s+-rf\s+\//,
  /rm\s+-rf\s+\*/,
  />\s*\/dev\/(sda|hda|nvme)/,
  /chmod\s+-R\s+777\s+\//,
  /dd\s+if=\/dev\/zero/,
  /:\(\)\s*\{.*\|.*\}/,  // fork bomb
  /sudo\s+rm/,
  /mkfs\./,
]
```

Module 16: Session Manager (`src/session/manager.ts`)

Owns `previous_response_id` threading — the most critical piece of state for WebSocket continuity.

```
class SessionManager {
  previousResponseId: string | null = null
  sessionId: string          // UUID generated at startup
  startedAt: number          // Timestamp
  model: string
  totalInputTokens: number = 0
```

```
  totalOutputTokens: number = 0

  updateResponseId(id: string): void   // Called on every turn_complete
  recordUsage(usage: TokenUsage): void
  async persistTurn(turn: ConversationTurn): Promise<void> // Atomic JSONL append
}
```

Transcript persistence (`src/session/persistence.ts`) — atomic JSONL writes (temp → rename pattern):

```
const tmp = `${filepath}.tmp.${Date.now()}`
await fs.writeFile(tmp, line + '\n', { flag: 'a' })
await fs.rename(tmp, filepath)
```

Git checkpoint (`src/session/checkpoint.ts`):

- Check `git diff --stat` after each completed turn
- If changes exist and 2+ minutes since last commit: auto-commit with a generated message
- Write `.claw/progress.md` before every full compaction
- Progress file format:

```
## Session Progress — {{timestamp}}
### Current Task: {{what the agent was doing}}
### Files Modified: {{git status output}}
### Key Decisions: {{architectural choices made}}
### Next Steps: {{what to do next}}
### Blocked On: {{any blockers}}
```

Preemptive reconnection (`src/session/reconnect.ts`):

- The Responses API WebSocket has a 60-minute connection limit
- Schedule reconnection at 55 minutes after `connectionOpenedAt`
- On reconnect: new `WebSocketTransport`, same `previous_response_id` → state preserved

Module 17: Terminal UI (`src/ui/`)

Non-negotiable: The REPL must be under 300 lines. No React. No JSX. No monolith. The 5,005-line `REPL.tsx` from the real Claude Code source is the anti-pattern to avoid.

REPL (`src/ui/repl.ts`):

- Use Node.js `readline` module for input
- Handle `/` commands locally without sending to model
- All rendering delegated to `renderer.ts` via a `UICallback` interface
- Ctrl-C: save session summary, close WebSocket, exit

`/` commands (never sent to model):

- `/compact` — trigger manual compaction
- `/tasks` — show todo list
- `/memory` — show memory file
- `/status` — session stats (tokens, model, connection state)
- `/model <name>` — switch model (reconnect with new model)
- `/mode manual|semi|auto` — change permission mode
- `/retry` — retry last failed turn
- `/undo` — pop shadow git stash (undo last apply_patch)
- `/clear` — clear conversation, preserve session
- `/quit` — exit with session summary

Renderer (`src/ui/renderer.ts`):

- Stream text deltas directly to `process.stdout.write()` character by character
- Tool call: `\x1b[33m$ ${command}\x1b[0m` (yellow)
- Tool result preview (2 lines max): `\x1b[90m${preview}\x1b[0m` (gray)
- Errors: `\x1b[31m[error] ${message}\x1b[0m` (red)
- Status: `\x1b[36m[claw] ${message}\x1b[0m` (cyan)

Diff display (`src/ui/diff-display.ts`) for `apply_patch` review:

```
┌─ src/auth/token.ts ──────────────────────────────┐
│ Refactor: extract token refresh into separate service │
├──────────────────────────────────────────────────────┤
│   23 |   async refreshToken() {                     │
│ - 24 |     const t = await fetch('/api/token')       │
│ + 24 |     const t = await this.tokenService.get()    │
└──────────────────────────────────────────────────────┘
Apply? [y/N/e]
```

Module 18: CLI Entry Point (`src/cli.ts`)

Eager construction (from the OpenDev paper): All modules initialized before the first user input. By the time the REPL prompt appears, everything is ready with zero first-call latency.

Startup sequence:

1. Load `.env`
2. Validate `OPENAI_API_KEY`
3. Initialize all modules (eager)
4. Connect WebSocket
5. Load CLAW.md hierarchy and build system prompt

6. Load skill index
7. Run memory search for project context
8. Show: `[claw v0.1.0] connected | model: ${model} | mode: ${mode} | tokens: 0`
9. Start REPL

Arguments:

```
claw [options] [prompt]

--model <name>       Model (default: gpt-4o)
--mode <mode>        manual | semi | auto (default: semi)
--max-tokens <n>     Context window size (default: 128000)
--no-memory          Disable memory for this session
--no-checkpoint      Disable git checkpoints
--verbose            Show all tool call details
--quiet              Show only final responses
-v, --version
-h, --help
```

Reasoning sandwich: Infer phase from model responses and vary the `reasoning.effort`:

- Planning phase (response mentions "approach", "plan", "I'll start by"): `effort: 'high'`
- Implementation phase: `effort: 'medium'`
- Verification phase (response mentions "test", "verify", "check"): `effort: 'high'`

Cross-Cutting Requirements

Prompt caching alignment: Static content (instructions, CLAW.md, skill index) always goes at the beginning of the `instructions` field and never changes mid-session. Dynamic content (tool results, user messages) goes in the `input` array. Never reorder existing messages — appending preserves the prefix property for caching.

Atomic writes everywhere: All file writes use temp → rename:

```
const tmp = `${filepath}.tmp.${Date.now()}`
await fs.writeFile(tmp, content, 'utf-8')
await fs.rename(tmp, filepath)
```

Error boundaries: Top-level `process.on('unhandledRejection', handler)`. Every async path has explicit `try/catch`. Never let an unhandled rejection crash the session.

AGENTS.md support: Support both `CLAW.md` (your native format) and `AGENTS.md` (OpenAI Codex standard, now adopted by Cursor, Copilot, Amp, Windsurf, Gemini CLI). Developers should not need separate config files for their conventions.

Implementation Order

Implement and verify in this exact sequence. Do not skip ahead.

1. `src/transport/circular-buffer.ts` — Foundation for replay
2. `src/transport/websocket-transport.ts` — Core connection (port from real source)
3. `src/types/responses-api.ts` — All event types
4. `src/engine/event-stream.ts` — Event parser
5. `src/engine/query-engine.ts` (minimal: just text response, no tools) — Verify you can connect to the Responses API WebSocket and get a text response
6. CHECKPOINT: Run `claw "what is 2+2?"` and verify WebSocket connection, streaming response, and `previous_response_id` threading work end-to-end
7. `src/tools/registry.ts` + `src/tools/shell.ts` — First tool
8. `src/tools/file-read.ts`, `src/tools/grep.ts`, `src/tools/glob.ts` — Read-only tools
9. `src/safety/danger-rules.ts` + `src/safety/permission-engine.ts`
10. `src/session/manager.ts` + `src/session/persistence.ts`
11. `src/ui/repl.ts` + `src/ui/renderer.ts` — Basic interactive REPL
12. CHECKPOINT: Full interactive session works, permission prompts work, transcripts are written
13. `src/tools/file-edit.ts` + `src/tools/file-write.ts` + `src/tools/apply-patch.ts`
14. `src/ui/diff-display.ts` — Patch review UI
15. `src/context/system-prompt-section.ts` + `src/context/prompt-builder.ts`
16. `src/context/compaction/micro-compact.ts`
17. `src/context/compaction/session-memory.ts`
18. `src/context/compaction/full-compact.ts` (ACC 5-stage)
19. `src/context/tool-result-shrinker.ts`
20. `src/memory/auto-memory.ts` + `src/memory/semantic-search.ts`
21. `src/memory/auto-dream.ts` (background)
22. `src/skills/loader.ts` + `src/skills/registry.ts`
23. `src/tools/skill-tool.ts` + `src/tools/memory-write.ts`
24. `src/engine/subagent.ts` + `src/transport/connection-pool.ts`
25. `src/tools/task.ts` — Subagent spawner tool
26. `src/engine/continuation.ts` — Self-healing
27. `src/engine/loop-detector.ts` — Anti-thrash
28. `src/session/checkpoint.ts` — Git checkpoints
29. `src/session/reconnect.ts` — Preemptive reconnect
30. `src/tools/todo-write.ts` + `src/ui/status-bar.ts`

31. `src/cli.ts` — Full CLI with eager construction
32. `tests/` — Write tests for: transport reconnect, compaction pipeline, memory retrieval, tool execution
33. `README.md` — Setup, usage, CLAW.md format, skill format

What NOT to Build

These are explicit exclusions:

- **No HTTP agent loop fallback** — WebSocket only. Reconnect if dropped.
- **No proxy or bridge** — Direct to `wss://api.openai.com/v1/responses`. No LiteLLM, Bifrost, or middleware.
- **No React/JSX in the terminal** — `repl.ts` must stay under 300 lines.
- **No ML classifier for skill selection** — LLM reads the frontmatter index and selects.
- **No synchronous file I/O** — All file ops use `fs/promises`.
- **No eager full skill body injection** — Only frontmatter at startup; full body on demand.

Quality Gates

Before marking any phase complete:

- [ ] WebSocket connects, `previous_response_id` threads across turns, no re-send of full history
- [ ] A 30-turn coding session completes without crash
- [ ] Compaction fires at 80%+ context fill and the agent continues coherently
- [ ] A subagent spawns on a separate connection, completes, returns summary, connection closes
- [ ] `apply_patch` correctly applies a unified diff and displays colorized review
- [ ] Memory written in session 1 is retrieved (semantic + exact) in session 2
- [ ] Skill body loads ONLY when `skill_tool` is called — NOT at startup
- [ ] Preemptive reconnect fires at 55 minutes without losing session state
- [ ] Danger rules block `rm -rf /` regardless of permission mode
- [ ] REPL handles Ctrl-C gracefully: session saved, WebSocket closed, exit

Environment Variables

```
# Required
OPENAI_API_KEY=sk-...

# Optional
CLAW_MODEL=gpt-4o
```

```
CLAW_MODE=semi
CLAW_MAX_TOKENS=128000
CLAW_DEBUG=false           # Log all WebSocket frames to .claw/debug.log
CLAW_NO_KEEPALIVE=false    # Disable keepalive frames (for testing)
```